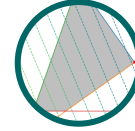


Continuous Optimization



Since machine learning algorithms are implemented on a computer, the mathematical formulations are expressed as numerical optimization methods. This chapter describes the basic numerical methods for training machine learning models. Training a machine learning model often boils down to finding a good set of parameters. The notion of “good” is determined by the objective function or the probabilistic model, which we will see examples of in the second part of this book. Given an objective function, finding the best value is done using optimization algorithms.

This chapter covers two main branches of continuous optimization (Figure 7.1): unconstrained and constrained optimization. We will assume in this chapter that our objective function is differentiable (see Chapter 5), hence we have access to a gradient at each location in the space to help us find the optimum value. By convention, most objective functions in machine learning are intended to be minimized, that is, the best value is the minimum value. Intuitively finding the best value is like finding the valleys of the objective function, and the gradients point us uphill. The idea is to move downhill (opposite to the gradient) and hope to find the deepest point. For unconstrained optimization, this is the only concept we need, but there are several design choices, which we discuss in Section 7.1. For constrained optimization, we need to introduce other concepts to manage the constraints (Section 7.2). We will also introduce a special class of problems (convex optimization problems in Section 7.3) where we can make statements about reaching the global optimum.

Consider the function in Figure 7.2. The function has a *global minimum* around $x = -4.5$, with a function value of approximately -47 . Since the function is “smooth,” the gradients can be used to help find the minimum by indicating whether we should take a step to the right or left. This assumes that we are in the correct bowl, as there exists another *local minimum* around $x = 0.7$. Recall that we can solve for all the stationary points of a function by calculating its derivative and setting it to zero. For

$$\ell(x) = x^4 + 7x^3 + 5x^2 - 17x + 3, \quad (7.1)$$

we obtain the corresponding gradient as

$$\frac{d\ell(x)}{dx} = 4x^3 + 21x^2 + 10x - 17. \quad (7.2)$$

Since we consider data and models in $\mathbb{R}^D$, the optimization problems we face are *continuous* optimization problems, as opposed to *combinatorial* optimization problems for discrete variables.

global minimum

local minimum

Stationary points are the real roots of the derivative, that is, points that have zero gradient.

$$(\arg) \min_{x \in \mathbb{R}^n} f(x)$$

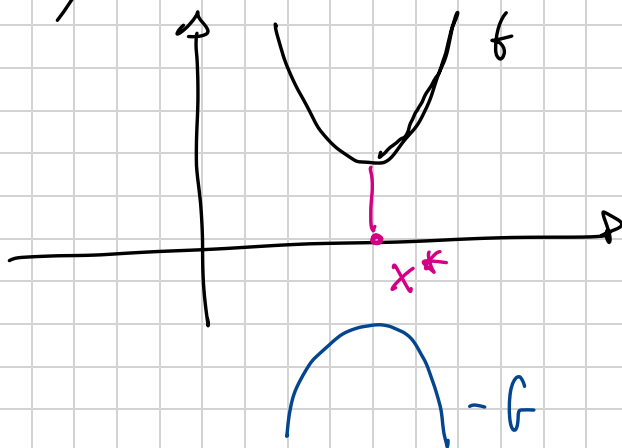
$$f: \mathbb{R}^n \rightarrow \mathbb{R}$$

$$x \in \mathbb{R}^n$$

$$\rightarrow \max f(x)$$

$$\arg \max_{x \in \mathbb{R}} f(x) =$$

$$= \arg \min_{x \in \mathbb{R}} f(x)$$



Definitions

- 1) x^* is a point of ^{strict} local minimum for f if it exists $\varepsilon > 0$:

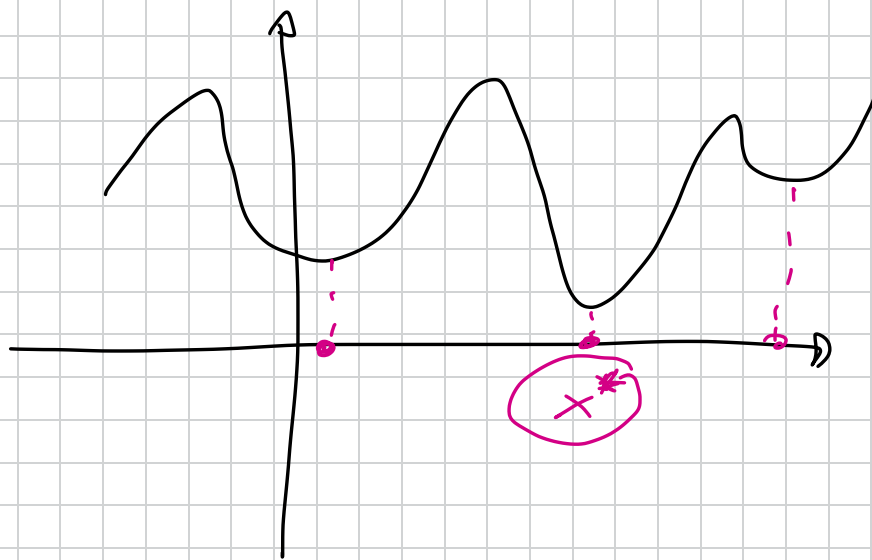
$$f(x^*) \leq f(x) \quad \forall x \in \mathbb{R}^n \text{ with } \|x - x^*\| < \varepsilon$$

$$f(x^*) < f(x)$$

- 2) x^* is a point of ^{strict} global minimum for f if

$$f(x^*) \leq f(x) \quad \forall x \in \mathbb{R}^n$$

$$f(x^*) < f(x) \quad \forall x \in \mathbb{R}^n$$



$\min_{x \in \mathbb{R}^n} f(x)$ unconstrained

$\min_{a \leq x \leq b} f(x)$ constrained

Optimality conditions

FIRST ORDER CONDITIONS

If x^* is a point of local minimum and f is differentiable with continuity around x^* , then

$$\nabla f(x^*) = 0$$

NECESSARY CONDITION

x^* LOCAL MINIMUM $\Rightarrow \nabla f(x^*) = 0$
 $\nLeftarrow$

$x^* : \nabla f(x^*) = 0$ called
 STATIONARY POINT

SECOND ORDER CONDITIONS

If x^* is a local minimum for f and f is twice differentiable around x^* , then

$$\nabla f(x^*) = 0 \text{ \& } \nabla^2 f(x^*).$$

is positive semidefinite

$$\nabla^2 f(x^*) \text{ symmetric matrix } n \times n$$

$$\nabla^2 f_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j} \quad i, j = 1 \dots n$$

$$\frac{\partial^2 f}{\partial x_j \partial x_i}$$

NECESSARY CONDITION

x^* LOCAL MINIMUM $\Rightarrow \nabla f(x^*) = 0$ \& $\nabla^2 f(x^*)$
is pos. semid.

SUFFICIENT CONDITION OF THE SECOND ORDER

If f is twice differentiable with continuity around x^* and $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*)$ is positive definite then

x^* is a strict local minimum of f .

SUFFICIENT CONDITION

$$\begin{array}{l} \nabla f(x^*) = 0 \text{ \& } \\ \nabla^2 f(x^*) \text{ pos def} \end{array} \Rightarrow x^* \text{ strict local minimum}$$

REMARK:

If $\nabla f(x^*) = 0$ then x^* can be:

- a) a minimum for f
- b) a maximum for f
- c) a saddle point for f

Example

$$f(x_1, x_2) = (x_1^2 + x_2^2 - 1)^2 + (x_2^2 - 1)^2$$

find the stationary points
(classify them)

$$\nabla f(x_1, x_2) = \left(\frac{\partial}{\partial x_1} (x_1^2 + x_2^2 - 1)^2, \frac{\partial}{\partial x_2} (x_1^2 + x_2^2 - 1)^2 + 2(x_2^2 - 1) \right)$$

$$\nabla f(x_1, x_2) = (0, 0)$$

$$\begin{cases} 2x_1(x_1^2 + x_2^2 - 1) = 0 \\ 2x_2(x_1^2 + x_2^2 - 1) + 4x_2(x_2^2 - 1) = 0 \end{cases}$$

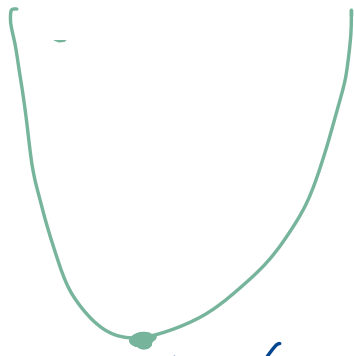
$$\nabla^2 f(x_1, x_2) \quad 2 \times 2 \text{ matrix}$$

$$\nabla^2 f(x_1^*, x_2^*) \quad (x_1^*, x_2^*) \text{ is}$$

a stationary point

→ is positive definite?

$\Rightarrow (x_1^*, x_2^*)$ strict local minimum



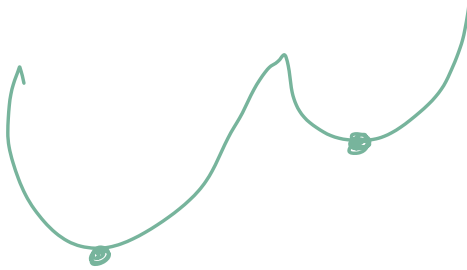
STRICT (LOCAL)



x^* LOCAL

$$f(x^*) = f(x)$$

$$x \in [x^* - \epsilon, x^* + \epsilon]$$



Def.

A function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is said to be differentiable at a point x_0 if there exists a linear mapping $J: \mathbb{R}^n \rightarrow \mathbb{R}^n$

$$\lim_{\underline{h} \rightarrow 0} \frac{\|f(\underline{x}_0 + \underline{h}) - f(\underline{x}_0) - J(\underline{h})\|}{\|\underline{h}\|} = 0$$

This means that f can be approximated by a linear mapping around x_0 .

f is differentiable in a domain D if it is differentiable in each point of D .

Proposition

If A is an open set of $\mathbb{R}^n$, $f: A \rightarrow \mathbb{R}$, and all the partial derivatives of f in x_0 , $\left(\frac{\partial f}{\partial x_i}(x_0), i=1 \dots n \right)$ exist and are continuous, then f is differentiable in x_0 .

N.B. NECESSARY CONDITION

$\frac{\partial f}{\partial x_i}(\underline{x}_0)$ exist and is continuous
 $\forall i = 1, \dots, n$



f is differentiable in $\underline{x}_0$

CONVEX FUNCTIONS

Def. CONVEX SET

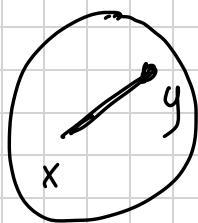
A set $C \subset \mathbb{R}^n$ is CONVEX if

$$\forall x, y \in C \text{ and } \forall \theta \in [0, 1] \subset \mathbb{R}$$

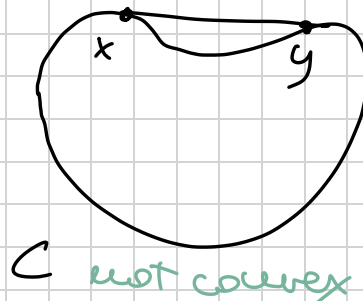
we have:

$$\rightarrow \theta x + (1 - \theta)y \in C$$

↓
equation of the line connecting
x and y



C
convex



C
not convex

Def. CONVEX FUNCTION

$f: C \subset \mathbb{R}^n \rightarrow \mathbb{R}$, C convex set

f is a convex function if

$$\forall x, y \in C \text{ and } \theta \in [0, 1] \subset \mathbb{R}$$

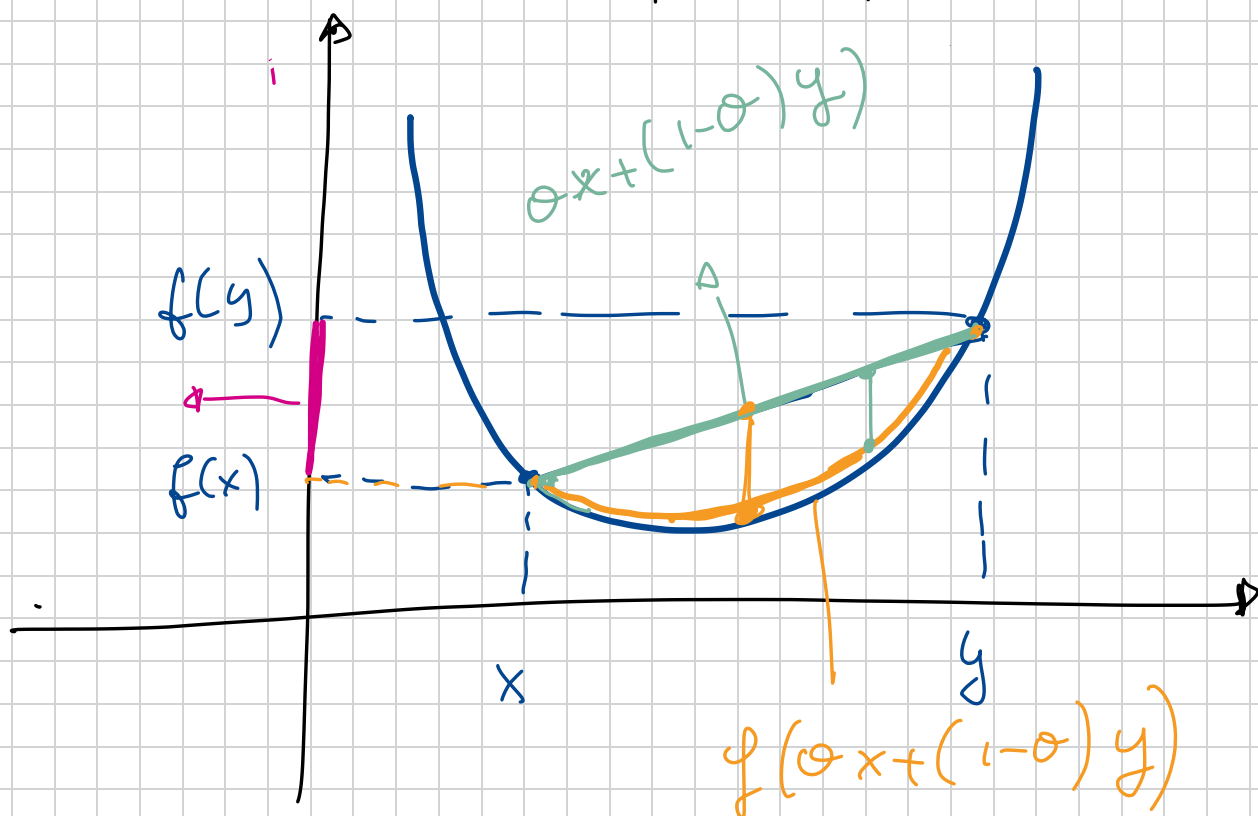
we have:

$$f(\underbrace{\theta x + (1-\theta)y}) \leq \underbrace{\theta f(x) + (1-\theta)f(y)}$$

$$\underbrace{\theta x + (1-\theta)y}_{z} \in C$$

line connecting $f(x)$ and $f(y)$

a line connecting x and y
lies above the function in x and y



Property

- If f is a convex function, then each local minimum is also a global minimum.
- If f is convex and differentiable then each stationary point is a global minimum of f .

$$\nabla f(x^*) = 0 \Rightarrow x^* \text{ global minimum}$$

$F: \|Ax - b\|_2^2 \rightarrow$ convex function

$$\|Ax - b\|_2^2: \mathbb{R}^n \rightarrow \mathbb{R}$$

$$\nabla F(x) = 2A^T A x - 2A^T b$$

$$\nabla F(x) = 0 \Rightarrow 2A^T A x - 2A^T b = 0$$

$$\Rightarrow A^T A x = A^T b \quad \text{LINEAR SYSTEM}$$

The solution x is the global minimum of F .

RESUME

$$\min_{x \in \mathbb{R}^m} \|Ax - b\|_2^2 \rightarrow \text{LEAST SQUARES PROBLEM}$$

if A has full rank has a unique solution:

1) by solving the linear system obtained from the first order conditions ($\nabla f(x) = 0$)

$$A^T A x = A^T b \quad (n \times n)$$

system of normal equations

$A^T A$ is symmetric & positive definite $\Rightarrow$ apply the

Cholesky decomposition

NOTE that A can be of size $n \times m$, $n \geq m$

$A^T A$ of size $m \times m$

For a general function $f: \mathbb{R}^n \rightarrow \mathbb{R}$,
how can we solve

$$\min_{x \in \mathbb{R}^n} f(x) \quad ?$$

→ ITERATIVE ALGORITHMS THAT
COMPUTE A LOCAL MINIMUM of f .

create a sequence of "tentative
solutions" of the problem

$$\underline{x}_0, \underline{x}_1, \underline{x}_2, \dots, \underline{x}_k, \underline{x}_{k+1}, \dots$$

with the property:

$$\lim_{k \rightarrow \infty} \underline{x}_k = x^*$$

x^* "SOLUTION" of the problem

Starting from a given x_0

$$x_k = \underline{G}(x_{k-1})$$

$$\left(x_k = G(\underbrace{x_{k-1}, x_{k-2}, \dots, x_{k-p}}_{\text{p previous iterates}}) \right) \leftarrow$$

$$\{x_0, x_1, \dots, x_{k^*}\} \dots$$

TRUNCATION ERROR

- k^* IS OBTAINED BY

STOPPING CRITERIA

$$\rightarrow \|\nabla f(x_k)\| < \tau \quad \tau = \text{tolerance}$$

A PARTICULAR CLASS OF ITERATIVE ALGORITHMS TO COMPUTE $\min$

$$\min_{x \in \mathbb{R}^n} f(x)$$

↓
objective function

DESCENT METHODS

$$x_k = G(x_{k-1}) \rightarrow$$

$$\rightarrow \underline{x_k = x_{k-1} + \alpha_{k-1} p_{k-1}} \leftarrow$$

x_{k-1} previous iterate $\in \mathbb{R}^n$

$p_{k-1} \in \mathbb{R}^n \rightarrow$ search direction

$\alpha_{k-1} \in \mathbb{R}^+ \rightarrow$ step length

$p_{k-1} \rightarrow$ also called descent direction why?

p_{k-1} is chosen so that:

$$f(x_k) < f(x_{k-1})$$

Definition of descent direction:

p is a descent direction in x for f if exists $\bar{\alpha} > 0$:

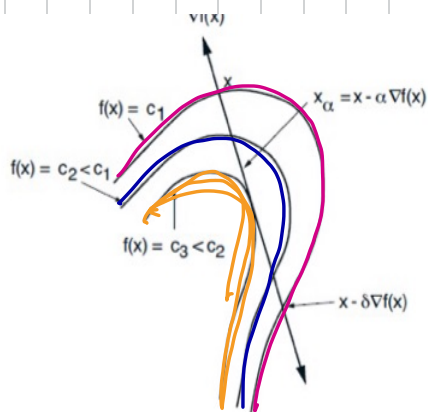
$$f(\underline{x + \alpha p}) < f(x) \quad \forall \alpha \in \underline{[0, \bar{\alpha}]}$$

$$x = x_{k-1} \quad x + \alpha p = x_k$$

level curve $\rightarrow \{x \in \mathbb{R}^n : f(x) = c\}$

:

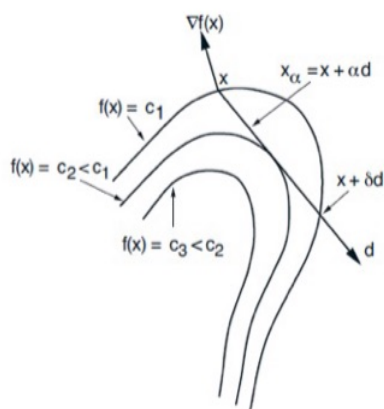
.



If $\nabla f(x) \neq 0$, there is an interval $(0, \delta)$ of stepsizes such that

$$f(x - \alpha \nabla f(x)) < f(x)$$

for all $\alpha \in (0, \delta)$.



If d makes an angle with $\nabla f(x)$ that is greater than 90 degrees,

$$\nabla f(x)'d < 0,$$

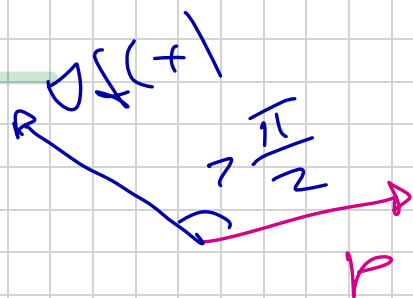
there is an interval $(0, \delta)$ of stepsizes such that $f(x + \alpha d) < f(x)$ for all $\alpha \in (0, \delta)$.

Property:

If f is continuous and differentiable in $\bar{x}$ and $p \in \mathbb{R}^n$, $p \neq 0$, so that:

$$p^T \cdot \nabla f(\bar{x}) < 0$$

then p is a descent direction for f in $\bar{x}$.



$p = -\nabla f(x) \Rightarrow p$ is a descent direction for f in $\bar{x}$.

Example

$$f(x) = x^2 + 3y^2 - 3xy$$

$$f: \mathbb{R}^2 \rightarrow \mathbb{R}$$

$$\bar{x} = (1, 1) \quad p = (1, 0)$$

Is p a descent direction for f in $\bar{x}$?

$$\nabla f(x) = (2x - 3y, 6y - 3x)$$

$$\nabla f(\bar{x}) = (-1, 3)$$

$$p \cdot \nabla f^T(\bar{x}) = (1, 0) \cdot \begin{pmatrix} -1 \\ 3 \end{pmatrix} = -1 < 0$$

p is a descent direction for f in $\bar{x}$.

- If $p \cdot \nabla f^T(\bar{x}) > 0 \Rightarrow p$ is not a descent direction for f in $\bar{x}$

- If $p \cdot \nabla f^T(\bar{x}) = 0 \Rightarrow p$ can be a descent direction or not.

$$\underline{x}_{k+1} = \underline{x}_k + \alpha_k \underline{p}_k$$

REMARK

If p is a descent direction for f in x_k , the choice α_k :

$$f(x_k + \alpha_k p_k) = f(x_{k+1}) < f(x_k)$$

is NOT SUFFICIENT TO GUARANTEE

that $\lim_{k \rightarrow \infty} \underline{x}_k = x^*$

HOW TO CHOOSE α_k (STEP LENGTH)

- Common choice in RL is α_k constant $\forall k$.

$$\alpha_k = \alpha$$

$\Rightarrow$ this does not guarantee the convergence.

- Inexact search

- a) α_k produces a "sufficient" decrease of f in x_k
- b) α_k produces a "sufficient" shift for x_{k-1} ($\|x_{k-1} - x_k\|$ sufficient)

BACKTRACKING PROCEDURE

- Set a "tentative value" $\bar{\alpha} = 1$
- Check if

$$\rightarrow f(\underbrace{x_k + \bar{\alpha} p_k}_{x_{k+1}}) \leq f(x_k) + c_1 \bar{\alpha} \underbrace{\nabla f(x_k)^T p_k}_{\leq 0}$$

$c_1 \in (0, 1]$

If not true $\rightarrow \bar{\alpha} = \bar{\alpha} \cdot \rho$

$$\rho = \frac{1}{2}$$

$$\rho < 1$$

Backtracking algorithm

Set $\alpha = \bar{\alpha} = 1$

Choose $\rho < 1, c_1 \in (0, 1)$

$$j = 0$$

while $f(x_k + \alpha p_k) > f(x_k) + c_1 \alpha \nabla f(x_k)^T p_k$

$$\alpha = \alpha \cdot \rho$$

$$j = j + 1$$

if $\alpha < \text{Tolerance } (\sim 10^{-8})$ stop
the algorithm

result: $\alpha_k = \alpha$

if $j = j^*$ then stop

Theorem CONVERGENCE

Consider a descent method

$$x_{k+1} = x_k + \alpha_k p_k$$

where α_k 's chosen by a
backtracking procedure.

Then:

$$\lim_{k \rightarrow \infty} x_k = x^*, \quad x^* \text{ local minimum.}$$

GRADIENT DESCENT ALGORITHM

Input: $f, \nabla f, x_0, \underline{\text{tolerances}}$

$k=0$

while stopping conditions are not

satisfied

$$\alpha < \begin{cases} \text{const. } \bar{\alpha} \\ \text{backtracking} \end{cases}$$

$$x = x_0 - \alpha_k \nabla f(x_0)$$

$$x_0 = x$$

$$k = k + 1$$

$$p = -\nabla f(x)$$

output: x

$$x^* \text{ local minimum} \Rightarrow \nabla f(x^*) = \underline{0}$$

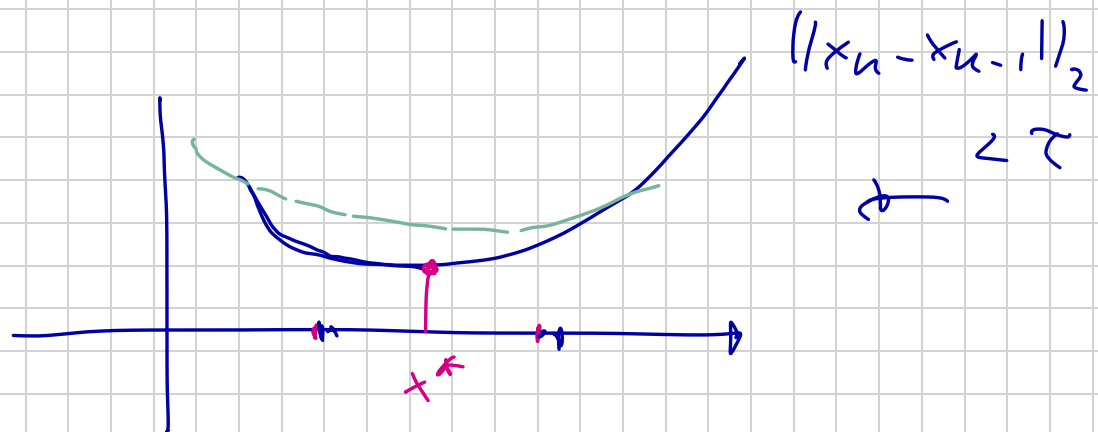
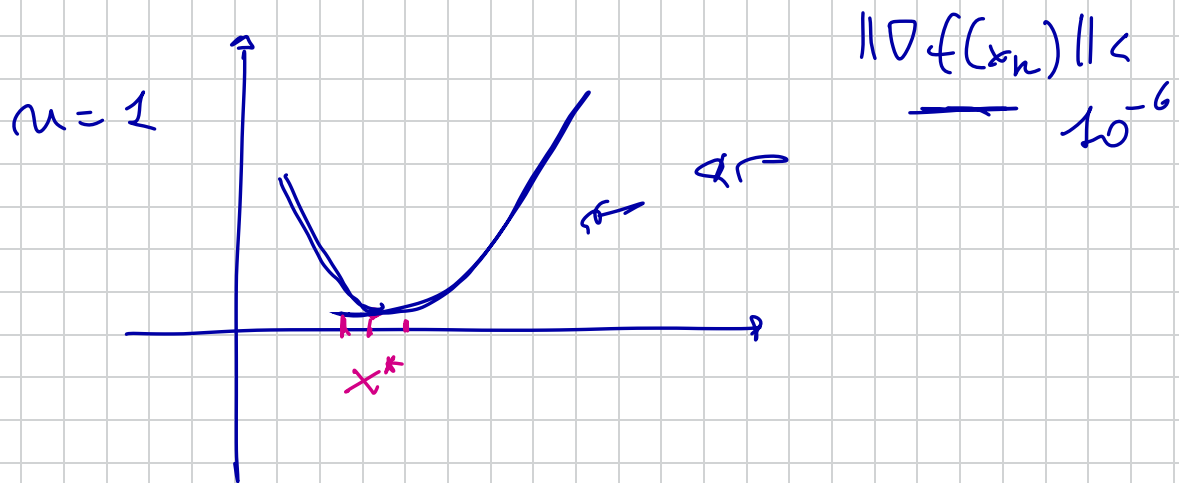
$\Downarrow$

$$1) \quad \|\nabla f(x_k)\| < \tau \quad (\text{absolute})$$

$$1b) \quad \frac{\|\nabla f(x_k)\|}{\|\nabla f(x_0)\|} < \tau_1 \quad (\text{relative})$$

→ 2) $\|x_k - x_{k-1}\| < \tau$ (absolute)

2b) $\frac{\|x_k - x_{k-1}\|}{\|x_k\|} < \tau_1$ (relative)



3) $K = \text{MAXIMUM NUMBER OF ITERATIONS}$

IT DOES NOT GUARANTEE THAT
THE SOLUTION IS CLOSE TO
THE MINIMUM.

HOW TO CHOOSE x_0

x_0 : influence the result →

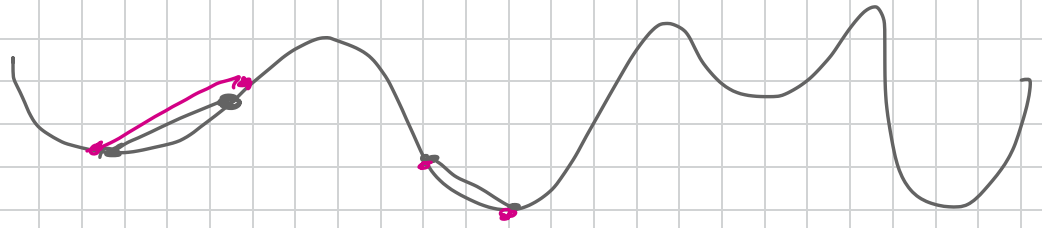
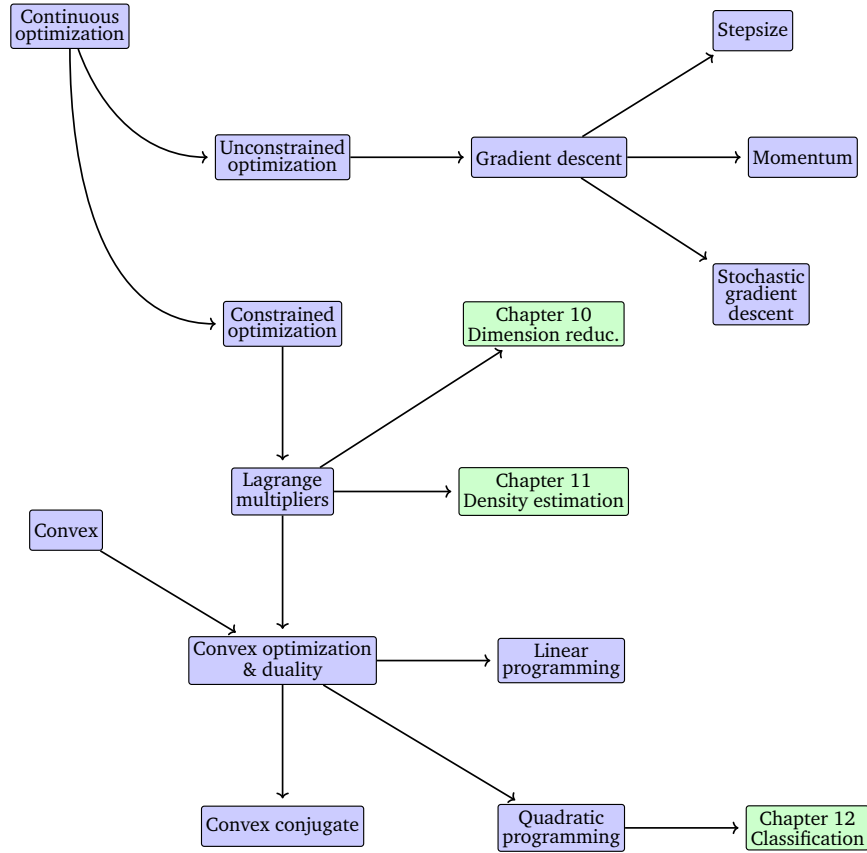


Figure 7.1 A mind map of the concepts related to optimization, as presented in this chapter. There are two main ideas: gradient descent and convex optimization.



Since this is a cubic equation, it has in general three solutions when set to zero. In the example, two of them are minimums and one is a maximum (around $x = -1.4$). To check whether a stationary point is a minimum or maximum, we need to take the derivative a second time and check whether the second derivative is positive or negative at the stationary point. In our case, the second derivative is

$$\frac{d^2\ell(x)}{dx^2} = 12x^2 + 42x + 10. \quad (7.3)$$

By substituting our visually estimated values of $x = -4.5, -1.4, 0.7$, we will observe that as expected the middle point is a maximum ($\frac{d^2\ell(x)}{dx^2} < 0$) and the other two stationary points are minimums.

Note that we have avoided analytically solving for values of x in the previous discussion, although for low-order polynomials such as the preceding we could do so. In general, we are unable to find analytic solutions, and hence we need to start at some value, say $x_0 = -6$, and follow the negative gradient. The negative gradient indicates that we should go

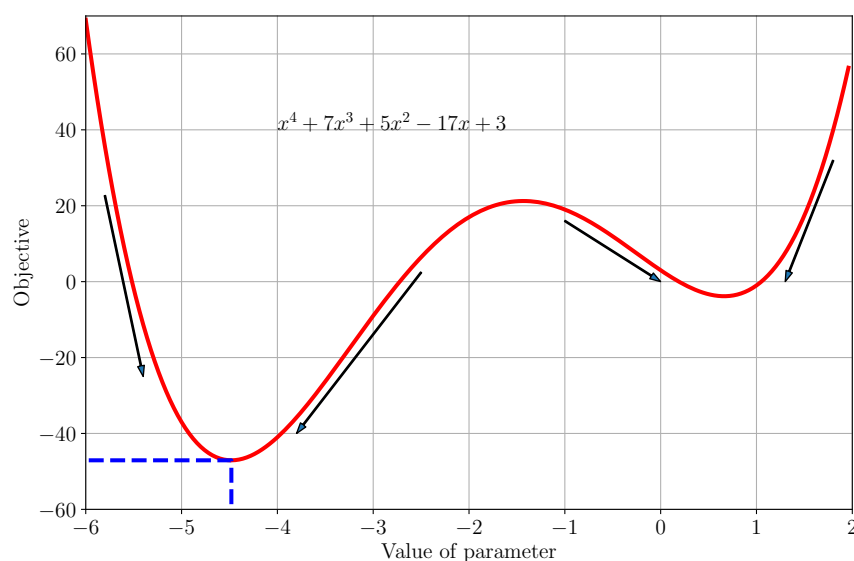


Figure 7.2 Example objective function. Negative gradients are indicated by arrows, and the global minimum is indicated by the dashed blue line.

right, but not how far (this is called the step-size). Furthermore, if we had started at the right side (e.g., $x_0 = 0$) the negative gradient would have led us to the wrong minimum. Figure 7.2 illustrates the fact that for $x > -1$, the negative gradient points toward the minimum on the right of the figure, which has a larger objective value.

In Section 7.3, we will learn about a class of functions, called convex functions, that do not exhibit this tricky dependency on the starting point of the optimization algorithm. For convex functions, all local minimums are global minimum. It turns out that many machine learning objective functions are designed such that they are convex, and we will see an example in Chapter 12.

The discussion in this chapter so far was about a one-dimensional function, where we are able to visualize the ideas of gradients, descent directions, and optimal values. In the rest of this chapter we develop the same ideas in high dimensions. Unfortunately, we can only visualize the concepts in one dimension, but some concepts do not generalize directly to higher dimensions, therefore some care needs to be taken when reading.

According to the Abel–Ruffini theorem, there is in general no algebraic solution for polynomials of degree 5 or more (Abel, 1826).

For convex functions all local minima are global minimum.

see notes

7.1 Optimization Using Gradient Descent

We now consider the problem of solving for the minimum of a real-valued function

$$\min_x f(x), \quad (7.4)$$

where $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is an objective function that captures the machine learning problem at hand. We assume that our function f is differentiable, and we are unable to analytically find a solution in closed form.

Gradient descent is a first-order optimization algorithm. To find a local minimum of a function using gradient descent, one takes steps proportional to the negative of the gradient of the function at the current point. Recall from Section 5.1 that the gradient points in the direction of the steepest ascent. Another useful intuition is to consider the set of lines where the function is at a certain value ($f(\mathbf{x}) = c$ for some value $c \in \mathbb{R}$), which are known as the contour lines. The gradient points in a direction that is orthogonal to the contour lines of the function we wish to optimize.

Let us consider multivariate functions. Imagine a surface (described by the function $f(\mathbf{x})$) with a ball starting at a particular location $\mathbf{x}_0$. When the ball is released, it will move downhill in the direction of steepest descent. Gradient descent exploits the fact that $f(\mathbf{x}_0)$ decreases fastest if one moves from $\mathbf{x}_0$ in the direction of the negative gradient $-((\nabla f)(\mathbf{x}_0))^\top$ of f at $\mathbf{x}_0$. We assume in this book that the functions are differentiable, and refer the reader to more general settings in Section 7.4. Then, if

$$\mathbf{x}_1 = \mathbf{x}_0 - \gamma((\nabla f)(\mathbf{x}_0))^\top \quad (7.5)$$

for a small *step-size* $\gamma \geq 0$, then $f(\mathbf{x}_1) \leq f(\mathbf{x}_0)$. Note that we use the transpose for the gradient since otherwise the dimensions will not work out.

This observation allows us to define a simple gradient descent algorithm: If we want to find a local optimum $f(\mathbf{x}_*)$ of a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $\mathbf{x} \mapsto f(\mathbf{x})$, we start with an initial guess $\mathbf{x}_0$ of the parameters we wish to optimize and then iterate according to

$$\mathbf{x}_{i+1} = \mathbf{x}_i - \gamma_i((\nabla f)(\mathbf{x}_i))^\top. \quad (7.6)$$

For suitable step-size γ_i , the sequence $f(\mathbf{x}_0) \geq f(\mathbf{x}_1) \geq \dots$ converges to a local minimum.

Example 7.1

Consider a quadratic function in two dimensions

$$f\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = \frac{1}{2} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^\top \begin{bmatrix} 2 & 1 \\ 1 & 20 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} - \begin{bmatrix} 5 \\ 3 \end{bmatrix}^\top \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad (7.7)$$

with gradient

$$\nabla f\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^\top \begin{bmatrix} 2 & 1 \\ 1 & 20 \end{bmatrix} - \begin{bmatrix} 5 \\ 3 \end{bmatrix}^\top. \quad (7.8)$$

Starting at the initial location $\mathbf{x}_0 = [-3, -1]^\top$, we iteratively apply (7.6) to obtain a sequence of estimates that converge to the minimum value

We use the convention of row vectors for gradients.

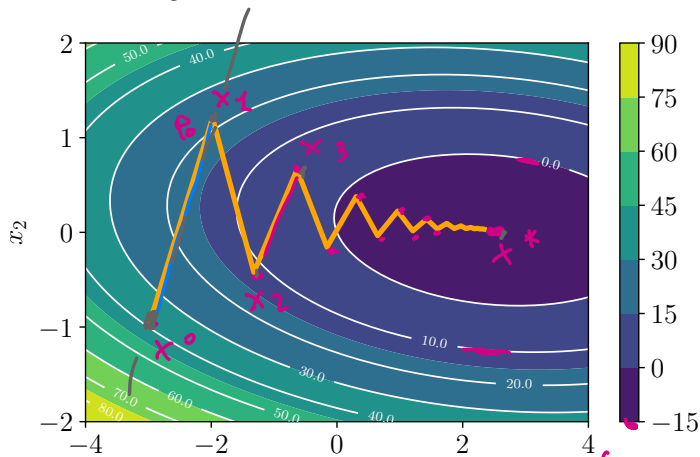


Figure 7.3 Gradient descent on a two-dimensional quadratic surface (shown as a heatmap). See Example 7.1 for a description.

CONTOUR LEVELS: $\{x \in \mathbb{R}^n : f(x) = c\}$

(illustrated in Figure 7.3). We can see (both from the figure and by plugging x_0 into (7.8) with $\gamma = 0.085$) that the negative gradient at x_0 points north and east, leading to $x_1 = [-1.98, 1.21]^\top$. Repeating that argument gives us $x_2 = [-1.32, -0.42]^\top$, and so on.

Remark. Gradient descent can be relatively slow close to the minimum: Its asymptotic rate of convergence is inferior to many other methods. Using the ball rolling down the hill analogy, when the surface is a long, thin valley, the problem is poorly conditioned (Trefethen and Bau III, 1997). For poorly conditioned convex problems, gradient descent increasingly “zigzags” as the gradients point nearly orthogonally to the shortest direction to a minimum point; see Figure 7.3. $\diamond$

7.1.1 Step-size

see notes

As mentioned earlier, choosing a good step-size is important in gradient descent. If the step-size is too small, gradient descent can be slow. If the step-size is chosen too large, gradient descent can overshoot, fail to converge, or even diverge. We will discuss the use of momentum in the next section. It is a method that smoothes out erratic behavior of gradient updates and dampens oscillations.

Adaptive gradient methods rescale the step-size at each iteration, depending on local properties of the function. There are two simple heuristics (Toussaint, 2012):

- When the function value increases after a gradient step, the step-size was too large. Undo the step and decrease the step-size.
- When the function value decreases the step could have been larger. Try to increase the step-size.

The step-size is also called the learning rate.

Although the “undo” step seems to be a waste of resources, using this heuristic guarantees monotonic convergence.

Example 7.2 (Solving a Linear Equation System)

When we solve linear equations of the form $Ax = b$, in practice we solve $Ax - b = 0$ approximately by finding x_* that minimizes the squared error

$$\|Ax - b\|^2 = (Ax - b)^\top (Ax - b) \quad (7.9)$$

if we use the Euclidean norm. The gradient of (7.9) with respect to x is

$$\nabla_x = 2(Ax - b)^\top A. \quad (7.10)$$

We can use this gradient directly in a gradient descent algorithm. However, for this particular special case, it turns out that there is an analytic solution, which can be found by setting the gradient to zero. We will see more on solving squared error problems in Chapter 9.

Remark. When applied to the solution of linear systems of equations $Ax = b$, gradient descent may converge slowly. The speed of convergence of gradient descent is dependent on the *condition number* $\kappa = \frac{\sigma(A)_{\max}}{\sigma(A)_{\min}}$, which is the ratio of the maximum to the minimum singular value (Section 4.5) of A . The condition number essentially measures the ratio of the most curved direction versus the least curved direction, which corresponds to our imagery that poorly conditioned problems are long, thin valleys: They are very curved in one direction, but very flat in the other. Instead of directly solving $Ax = b$, one could instead solve $P^{-1}(Ax - b) = 0$, where P is called the *preconditioner*. The goal is to design P^{-1} such that $P^{-1}A$ has a better condition number, but at the same time P^{-1} is easy to compute. For further information on gradient descent, preconditioning, and convergence we refer to Boyd and Vandenberghe (2004, chapter 9). $\diamond$

condition number

preconditioner

7.1.2 Gradient Descent With Momentum

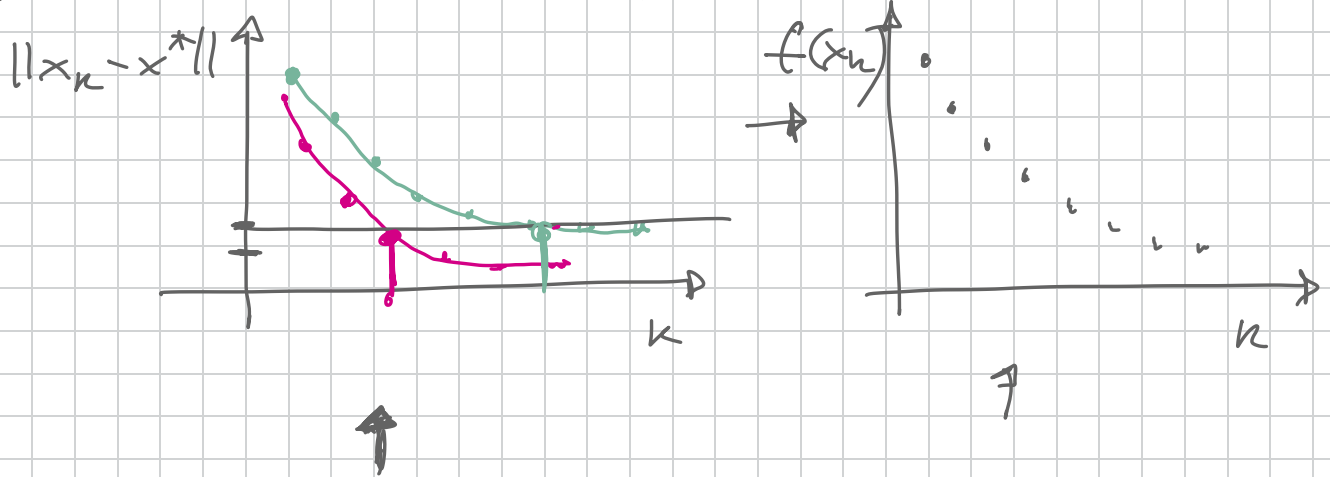
As illustrated in Figure 7.3, the convergence of gradient descent may be very slow if the curvature of the optimization surface is such that there are regions that are poorly scaled. The curvature is such that the gradient descent steps hops between the walls of the valley and approaches the optimum in small steps. The proposed tweak to improve convergence is to give gradient descent some memory.

Gradient descent with momentum (Rumelhart et al., 1986) is a method that introduces an additional term to remember what happened in the previous iteration. This memory dampens oscillations and smoothes out the gradient updates. Continuing the ball analogy, the momentum term emulates the phenomenon of a heavy ball that is reluctant to change directions. The idea is to have a gradient update with memory to implement

Goh (2017) wrote an intuitive blog post on gradient descent with momentum.

CONVERGENCE SPEED

→ Evaluate how fast the algorithm approximates the solution.



The method has Q convergence speed if:

$$\|x_k - x^*\| < C \underbrace{\|x_{k-1} - x^*\|}_{}^Q, \quad k > \bar{k}$$

$$0 < Q$$

1) IF $Q=1 \Rightarrow$ LINEAR CONVERGENCE

$$C < 1$$

2) IF $1 < Q < 2 \Rightarrow$ SUPER LINEAR CONVERGENCE

3) IF $Q=2 \Rightarrow$ QUADRATIC CONVERGENCE

a moving average. The momentum-based method remembers the update $\Delta \mathbf{x}_i$ at each iteration i and determines the next update as a linear combination of the current and previous gradients

$$\mathbf{x}_{i+1} = \mathbf{x}_i - \gamma_i ((\nabla f)(\mathbf{x}_i))^{\top} + \alpha \Delta \mathbf{x}_i \quad (7.11)$$

$= \mathbf{x}_i - \gamma_i \nabla f + \alpha (\mathbf{x}_i - \mathbf{x}_{i-1})$

$$\Delta \mathbf{x}_i = \mathbf{x}_i - \mathbf{x}_{i-1} = \alpha \Delta \mathbf{x}_{i-1} - \gamma_{i-1} ((\nabla f)(\mathbf{x}_{i-1}))^{\top}, \quad (7.12)$$

where $\alpha \in [0, 1]$. Sometimes we will only know the gradient approximately. In such cases, the momentum term is useful since it averages out different noisy estimates of the gradient. One particularly useful way to obtain an approximate gradient is by using a stochastic approximation, which we discuss next.

$\gamma_i \rightarrow \text{step length}$

7.1.3 Stochastic Gradient Descent

Computing the gradient can be very time consuming. However, often it is possible to find a “cheap” approximation of the gradient. Approximating the gradient is still useful as long as it points in roughly the same direction as the true gradient.

stochastic gradient
descent

Stochastic gradient descent (often shortened as SGD) is a stochastic approximation of the gradient descent method for minimizing an objective function that is written as a sum of differentiable functions. The word stochastic here refers to the fact that we acknowledge that we do not know the gradient precisely, but instead only know a noisy approximation to it. By constraining the probability distribution of the approximate gradients, we can still theoretically guarantee that SGD will converge.

In machine learning, given $n = 1, \dots, N$ data points, we often consider objective functions that are the sum of the losses L_n incurred by each example n . In mathematical notation, we have the form

$$L(\theta) = \sum_{n=1}^N L_n(\theta),$$

linear least squares
 $\|\Phi\theta - y\|_2^2 =$

(7.13)

where θ is the vector of parameters of interest, i.e., we want to find θ that minimizes L . An example from regression (Chapter 9) is the negative log-likelihood, which is expressed as a sum over log-likelihoods of individual examples so that

$$L(\theta) = - \sum_{n=1}^N \log p(y_n | \mathbf{x}_n, \theta), \quad (7.14)$$

where $\mathbf{x}_n \in \mathbb{R}^D$ are the training inputs, y_n are the training targets, and θ are the parameters of the regression model.

Standard gradient descent, as introduced previously, is a “batch” optimization method, i.e., optimization is performed using the full training set

$$\min_{\theta} L(\theta) = \min_{\theta} \sum_{n=1}^N L_n(\theta)$$

$$\begin{aligned} \nabla L(\theta) &= \nabla \sum_{n=1}^N L_n(\theta) = \\ &= \sum_{n=1}^N \nabla L_n(\theta) \end{aligned}$$

$$\nabla L(\theta) \approx \sum_{n \in S} \nabla L_n(\theta)$$

$$S = \{ \underbrace{n_1, n_2, \dots, n_k}_{N=100, \quad k=5} \} \subset \{1, 2, \dots, N\}$$

S MINIBATCH of size k .

$$\nabla L_{n_1}(\theta) + \nabla L_{n_2}(\theta) \dots + \nabla L_{n_k}(\theta)$$

- At each iteration, we choose a different minibatch of the same size.

$$n=10$$

$$k=1$$

$$k=5$$

→ EPOCH → number of iterations to select all the values $n \in \{1, \dots, N\}$
 STEP LENGTH α_k → LEARNING RATE

by updating the vector of parameters according to

$$\boldsymbol{\theta}_{i+1} = \boldsymbol{\theta}_i - \gamma_i (\nabla L(\boldsymbol{\theta}_i))^\top = \boldsymbol{\theta}_i - \gamma_i \sum_{n=1}^N (\nabla L_n(\boldsymbol{\theta}_i))^\top \quad (7.15)$$

classical
GD

for a suitable step-size parameter γ_i . Evaluating the sum gradient may require expensive evaluations of the gradients from all individual functions L_n . When the training set is enormous and/or no simple formulas exist, evaluating the sums of gradients becomes very expensive.

Consider the term $\sum_{n=1}^N (\nabla L_n(\boldsymbol{\theta}_i))$ in (7.15), we can reduce the amount of computation by taking a sum over a smaller set of L_n . In contrast to batch gradient descent, which uses all L_n for $n = 1, \dots, N$, we randomly choose a subset of L_n for mini-batch gradient descent. In the extreme case, we randomly select only a single L_n to estimate the gradient. The key insight about why taking a subset of data is sensible is to realize that for gradient descent to converge, we only require that the gradient is an unbiased estimate of the true gradient. In fact the term $\sum_{n=1}^N (\nabla L_n(\boldsymbol{\theta}_i))$ in (7.15) is an empirical estimate of the expected value (Section 6.4.1) of the gradient. Therefore, any other unbiased empirical estimate of the expected value, for example using any subsample of the data, would suffice for convergence of gradient descent.

Remark. When the learning rate decreases at an appropriate rate, and subject to relatively mild assumptions, stochastic gradient descent converges almost surely to local minimum (Bottou, 1998). $\diamond$

Why should one consider using an approximate gradient? A major reason is practical implementation constraints, such as the size of central processing unit (CPU)/graphics processing unit (GPU) memory or limits on computational time. We can think of the size of the subset used to estimate the gradient in the same way that we thought of the size of a sample when estimating empirical means (Section 6.4.1). Large mini-batch sizes will provide accurate estimates of the gradient, reducing the variance in the parameter update. Furthermore, large mini-batches take advantage of highly optimized matrix operations in vectorized implementations of the cost and gradient. The reduction in variance leads to more stable convergence, but each gradient calculation will be more expensive.

In contrast, small mini-batches are quick to estimate. If we keep the mini-batch size small, the noise in our gradient estimate will allow us to get out of some bad local optima, which we may otherwise get stuck in. In machine learning, optimization methods are used for training by minimizing an objective function on the training data, but the overall goal is to improve generalization performance (Chapter 8). Since the goal in machine learning does not necessarily need a precise estimate of the minimum of the objective function, approximate gradients using mini-batch approaches have been widely used. Stochastic gradient descent is very effective in large-scale machine learning problems (Bottou et al., 2018),

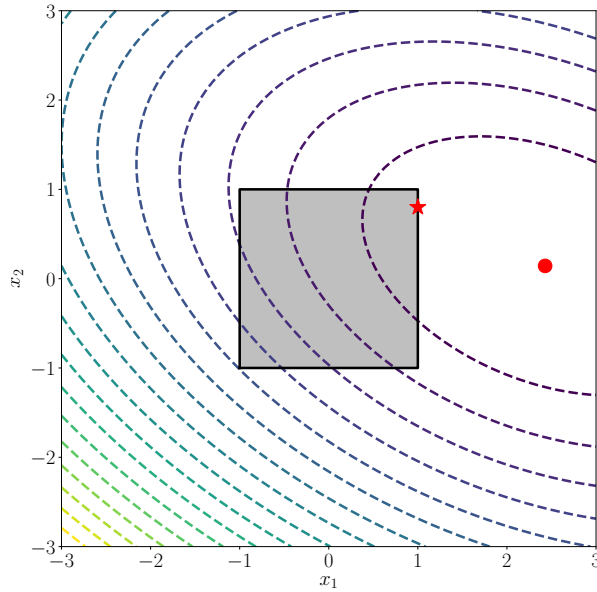


Figure 7.4
Illustration of constrained optimization. The unconstrained problem (indicated by the contour lines) has a minimum on the right side (indicated by the circle). The box constraints ($-1 \leq x \leq 1$ and $-1 \leq y \leq 1$) require that the optimal solution is within the box, resulting in an optimal value indicated by the star.

such as training deep neural networks on millions of images (Dean et al., 2012), topic models (Hoffman et al., 2013), reinforcement learning (Mnih et al., 2015), or training of large-scale Gaussian process models (Hensman et al., 2013; Gal et al., 2014).

7.2 Constrained Optimization and Lagrange Multipliers

In the previous section, we considered the problem of solving for the minimum of a function

$$\min_{\mathbf{x}} f(\mathbf{x}), \quad (7.16)$$

where $f : \mathbb{R}^D \rightarrow \mathbb{R}$.

In this section, we have additional constraints. That is, for real-valued functions $g_i : \mathbb{R}^D \rightarrow \mathbb{R}$ for $i = 1, \dots, m$, we consider the constrained optimization problem (see Figure 7.4 for an illustration)

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & g_i(\mathbf{x}) \leq 0 \quad \text{for all } i = 1, \dots, m. \end{aligned} \quad (7.17)$$

It is worth pointing out that the functions f and g_i could be non-convex in general, and we will consider the convex case in the next section.

One obvious, but not very practical, way of converting the constrained problem (7.17) into an unconstrained one is to use an indicator function

$$J(\mathbf{x}) = f(\mathbf{x}) + \sum_{i=1}^m \mathbf{1}(g_i(\mathbf{x})), \quad (7.18)$$

where $\mathbf{1}(z)$ is an infinite step function

$$\mathbf{1}(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ \infty & \text{otherwise} \end{cases}. \quad (7.19)$$

This gives infinite penalty if the constraint is not satisfied, and hence would provide the same solution. However, this infinite step function is equally difficult to optimize. We can overcome this difficulty by introducing *Lagrange multipliers*. The idea of Lagrange multipliers is to replace the step function with a linear function.

Lagrange multiplier

Lagrangian

We associate to problem (7.17) the *Lagrangian* by introducing the Lagrange multipliers $\lambda_i \geq 0$ corresponding to each inequality constraint respectively (Boyd and Vandenberghe, 2004, chapter 4) so that

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \sum_{i=1}^m \lambda_i g_i(\mathbf{x}) \quad (7.20a)$$

$$= f(\mathbf{x}) + \boldsymbol{\lambda}^\top \mathbf{g}(\mathbf{x}), \quad (7.20b)$$

where in the last line we have concatenated all constraints $g_i(\mathbf{x})$ into a vector $\mathbf{g}(\mathbf{x})$, and all the Lagrange multipliers into a vector $\boldsymbol{\lambda} \in \mathbb{R}^m$.

We now introduce the idea of Lagrangian duality. In general, duality in optimization is the idea of converting an optimization problem in one set of variables $\mathbf{x}$ (called the primal variables), into another optimization problem in a different set of variables $\boldsymbol{\lambda}$ (called the dual variables). We introduce two different approaches to duality: In this section, we discuss Lagrangian duality; in Section 7.3.3, we discuss Legendre-Fenchel duality.

Definition 7.1. The problem in (7.17)

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & g_i(\mathbf{x}) \leq 0 \quad \text{for all } i = 1, \dots, m \end{aligned} \quad (7.21)$$

primal problem

Lagrangian dual problem

is known as the *primal problem*, corresponding to the primal variables $\mathbf{x}$. The associated *Lagrangian dual problem* is given by

$$\begin{aligned} \max_{\boldsymbol{\lambda} \in \mathbb{R}^m} \quad & \mathfrak{D}(\boldsymbol{\lambda}) \\ \text{subject to} \quad & \boldsymbol{\lambda} \geq \mathbf{0}, \end{aligned} \quad (7.22)$$

where $\boldsymbol{\lambda}$ are the dual variables and $\mathfrak{D}(\boldsymbol{\lambda}) = \min_{\mathbf{x} \in \mathbb{R}^d} \mathcal{L}(\mathbf{x}, \boldsymbol{\lambda})$.

Remark. In the discussion of Definition 7.1, we use two concepts that are also of independent interest (Boyd and Vandenberghe, 2004).

minimax inequality

First is the *minimax inequality*, which says that for any function with two arguments $\varphi(\mathbf{x}, \mathbf{y})$, the maximin is less than the minimax, i.e.,

$$\max_{\mathbf{y}} \min_{\mathbf{x}} \varphi(\mathbf{x}, \mathbf{y}) \leq \min_{\mathbf{x}} \max_{\mathbf{y}} \varphi(\mathbf{x}, \mathbf{y}). \quad (7.23)$$

This inequality can be proved by considering the inequality

$$\text{For all } \mathbf{x}, \mathbf{y} \quad \min_{\mathbf{x}} \varphi(\mathbf{x}, \mathbf{y}) \leq \max_{\mathbf{y}} \varphi(\mathbf{x}, \mathbf{y}). \quad (7.24)$$

Note that taking the maximum over $\mathbf{y}$ of the left-hand side of (7.24) maintains the inequality since the inequality is true for all $\mathbf{y}$. Similarly, we can take the minimum over $\mathbf{x}$ of the right-hand side of (7.24) to obtain (7.23).

The second concept is *weak duality*, which uses (7.23) to show that primal values are always greater than or equal to dual values. This is described in more detail in (7.27). $\diamond$

Recall that the difference between $J(\mathbf{x})$ in (7.18) and the Lagrangian in (7.20b) is that we have relaxed the indicator function to a linear function. Therefore, when $\lambda \geq 0$, the Lagrangian $\mathcal{L}(\mathbf{x}, \lambda)$ is a lower bound of $J(\mathbf{x})$. Hence, the maximum of $\mathcal{L}(\mathbf{x}, \lambda)$ with respect to λ is

$$J(\mathbf{x}) = \max_{\lambda \geq 0} \mathcal{L}(\mathbf{x}, \lambda). \quad (7.25)$$

Recall that the original problem was minimizing $J(\mathbf{x})$,

$$\min_{\mathbf{x} \in \mathbb{R}^d} \max_{\lambda \geq 0} \mathcal{L}(\mathbf{x}, \lambda). \quad (7.26)$$

By the minimax inequality (7.23), it follows that swapping the order of the minimum and maximum results in a smaller value, i.e.,

$$\min_{\mathbf{x} \in \mathbb{R}^d} \max_{\lambda \geq 0} \mathcal{L}(\mathbf{x}, \lambda) \geq \max_{\lambda \geq 0} \min_{\mathbf{x} \in \mathbb{R}^d} \mathcal{L}(\mathbf{x}, \lambda). \quad (7.27)$$

This is also known as *weak duality*. Note that the inner part of the right-hand side is the dual objective function $\mathfrak{D}(\lambda)$ and the definition follows.

In contrast to the original optimization problem, which has constraints, $\min_{\mathbf{x} \in \mathbb{R}^d} \mathcal{L}(\mathbf{x}, \lambda)$ is an unconstrained optimization problem for a given value of λ . If solving $\min_{\mathbf{x} \in \mathbb{R}^d} \mathcal{L}(\mathbf{x}, \lambda)$ is easy, then the overall problem is easy to solve. We can see this by observing from (7.20b) that $\mathcal{L}(\mathbf{x}, \lambda)$ is affine with respect to λ . Therefore $\min_{\mathbf{x} \in \mathbb{R}^d} \mathcal{L}(\mathbf{x}, \lambda)$ is a pointwise minimum of affine functions of λ , and hence $\mathfrak{D}(\lambda)$ is concave even though $f(\cdot)$ and $g_i(\cdot)$ may be nonconvex. The outer problem, maximization over λ , is the maximum of a concave function and can be efficiently computed.

Assuming $f(\cdot)$ and $g_i(\cdot)$ are differentiable, we find the Lagrange dual problem by differentiating the Lagrangian with respect to $\mathbf{x}$, setting the differential to zero, and solving for the optimal value. We will discuss two concrete examples in Sections 7.3.1 and 7.3.2, where $f(\cdot)$ and $g_i(\cdot)$ are convex.

Remark (Equality Constraints). Consider (7.17) with additional equality constraints

$$\begin{aligned} & \min_{\mathbf{x}} \quad f(\mathbf{x}) \\ & \text{subject to} \quad g_i(\mathbf{x}) \leq 0 \quad \text{for all } i = 1, \dots, m \\ & \quad \quad \quad h_j(\mathbf{x}) = 0 \quad \text{for all } j = 1, \dots, n. \end{aligned} \quad (7.28)$$

We can model equality constraints by replacing them with two inequality constraints. That is for each equality constraint $h_j(\mathbf{x}) = 0$ we equivalently replace it by two constraints $h_j(\mathbf{x}) \leq 0$ and $h_j(\mathbf{x}) \geq 0$. It turns out that the resulting Lagrange multipliers are then unconstrained.

Therefore, we constrain the Lagrange multipliers corresponding to the inequality constraints in (7.28) to be non-negative, and leave the Lagrange multipliers corresponding to the equality constraints unconstrained. $\diamond$

7.3 Convex Optimization

We focus our attention of a particularly useful class of optimization problems, where we can guarantee global optimality. When $f(\cdot)$ is a convex function, and when the constraints involving $g(\cdot)$ and $h(\cdot)$ are convex sets, this is called a *convex optimization problem*. In this setting, we have *strong duality*: The optimal solution of the dual problem is the same as the optimal solution of the primal problem. The distinction between convex functions and convex sets are often not strictly presented in machine learning literature, but one can often infer the implied meaning from context.

convex optimization
problem
strong duality

convex set

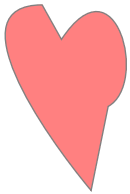
Definition 7.2. A set $\mathcal{C}$ is a *convex set* if for any $x, y \in \mathcal{C}$ and for any scalar θ with $0 \leq \theta \leq 1$, we have

$$\theta x + (1 - \theta)y \in \mathcal{C}. \quad (7.29)$$

Figure 7.5 Example of a convex set.



Figure 7.6 Example of a nonconvex set.



convex function
concave function

epigraph

Convex sets are sets such that a straight line connecting any two elements of the set lie inside the set. Figures 7.5 and 7.6 illustrate convex and nonconvex sets, respectively.

Convex functions are functions such that a straight line between any two points of the function lie above the function. Figure 7.2 shows a non-convex function, and Figure 7.3 shows a convex function. Another convex function is shown in Figure 7.7.

Definition 7.3. Let function $f : \mathbb{R}^D \rightarrow \mathbb{R}$ be a function whose domain is a convex set. The function f is a *convex function* if for all $\mathbf{x}, \mathbf{y}$ in the domain of f , and for any scalar θ with $0 \leq \theta \leq 1$, we have

$$f(\theta \mathbf{x} + (1 - \theta)\mathbf{y}) \leq \theta f(\mathbf{x}) + (1 - \theta)f(\mathbf{y}). \quad (7.30)$$

Remark. A *concave function* is the negative of a convex function. $\diamond$

The constraints involving $g(\cdot)$ and $h(\cdot)$ in (7.28) truncate functions at a scalar value, resulting in sets. Another relation between convex functions and convex sets is to consider the set obtained by “filling in” a convex function. A convex function is a bowl-like object, and we imagine pouring water into it to fill it up. This resulting filled-in set, called the *epigraph* of the convex function, is a convex set.

If a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is differentiable, we can specify convexity in

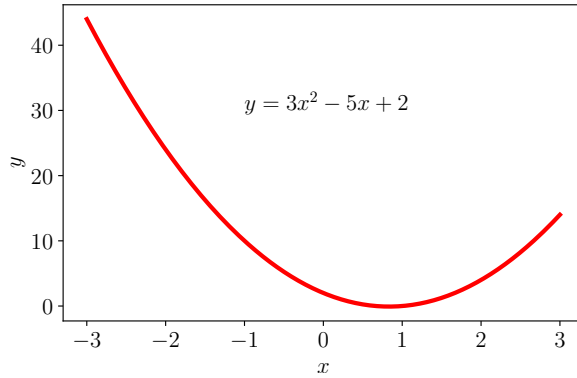


Figure 7.7 Example of a convex function.

terms of its gradient $\nabla_x f(x)$ (Section 5.2). A function $f(x)$ is convex if and only if for any two points x, y it holds that

$$f(y) \geq f(x) + \nabla_x f(x)^\top (y - x). \quad (7.31)$$

If we further know that a function $f(x)$ is twice differentiable, that is, the Hessian (5.147) exists for all values in the domain of x , then the function $f(x)$ is convex if and only if $\nabla_x^2 f(x)$ is positive semidefinite (Boyd and Vandenberghe, 2004).

Example 7.3

The negative entropy $f(x) = x \log_2 x$ is convex for $x > 0$. A visualization of the function is shown in Figure 7.8, and we can see that the function is convex. To illustrate the previous definitions of convexity, let us check the calculations for two points $x = 2$ and $x = 4$. Note that to prove convexity of $f(x)$ we would need to check for all points $x \in \mathbb{R}$.

Recall Definition 7.3. Consider a point midway between the two points (that is $\theta = 0.5$); then the left-hand side is $f(0.5 \cdot 2 + 0.5 \cdot 4) = 3 \log_2 3 \approx 4.75$. The right-hand side is $0.5(2 \log_2 2) + 0.5(4 \log_2 4) = 1 + 4 = 5$. And therefore the definition is satisfied.

Since $f(x)$ is differentiable, we can alternatively use (7.31). Calculating the derivative of $f(x)$, we obtain

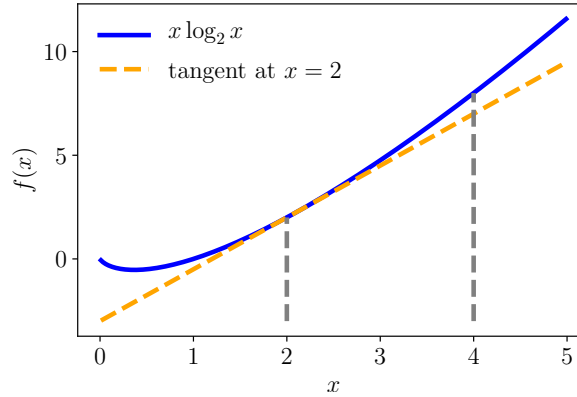
$$\nabla_x (x \log_2 x) = 1 \cdot \log_2 x + x \cdot \frac{1}{x \log_e 2} = \log_2 x + \frac{1}{\log_e 2}. \quad (7.32)$$

Using the same two test points $x = 2$ and $x = 4$, the left-hand side of (7.31) is given by $f(4) = 8$. The right-hand side is

$$f(x) + \nabla_x^\top (y - x) = f(2) + \nabla f(2) \cdot (4 - 2) \quad (7.33a)$$

$$= 2 + \left(1 + \frac{1}{\log_e 2}\right) \cdot 2 \approx 6.9. \quad (7.33b)$$

Figure 7.8 The negative entropy function (which is convex) and its tangent at $x = 2$.



We can check that a function or set is convex from first principles by recalling the definitions. In practice, we often rely on operations that preserve convexity to check that a particular function or set is convex. Although the details are vastly different, this is again the idea of closure that we introduced in Chapter 2 for vector spaces.

Example 7.4

A nonnegative weighted sum of convex functions is convex. Observe that if f is a convex function, and $\alpha \geq 0$ is a nonnegative scalar, then the function αf is convex. We can see this by multiplying α to both sides of the equation in Definition 7.3, and recalling that multiplying a nonnegative number does not change the inequality.

If f_1 and f_2 are convex functions, then we have by the definition

$$f_1(\theta x + (1 - \theta)y) \leq \theta f_1(x) + (1 - \theta)f_1(y) \quad (7.34)$$

$$f_2(\theta x + (1 - \theta)y) \leq \theta f_2(x) + (1 - \theta)f_2(y). \quad (7.35)$$

Summing up both sides gives us

$$\begin{aligned} & f_1(\theta x + (1 - \theta)y) + f_2(\theta x + (1 - \theta)y) \\ & \leq \theta f_1(x) + (1 - \theta)f_1(y) + \theta f_2(x) + (1 - \theta)f_2(y), \end{aligned} \quad (7.36)$$

where the right-hand side can be rearranged to

$$\theta(f_1(x) + f_2(x)) + (1 - \theta)(f_1(y) + f_2(y)), \quad (7.37)$$

completing the proof that the sum of convex functions is convex.

Combining the preceding two facts, we see that $\alpha f_1(x) + \beta f_2(x)$ is convex for $\alpha, \beta \geq 0$. This closure property can be extended using a similar argument for nonnegative weighted sums of more than two convex functions.

Remark. The inequality in (7.30) is sometimes called *Jensen's inequality*. In fact, a whole class of inequalities for taking nonnegative weighted sums of convex functions are all called Jensen's inequality. $\diamond$

Jensen's inequality

In summary, a constrained optimization problem is called a *convex optimization problem* if

convex optimization problem

$$\begin{aligned} & \min_{\mathbf{x}} f(\mathbf{x}) \\ & \text{subject to } g_i(\mathbf{x}) \leq 0 \quad \text{for all } i = 1, \dots, m \\ & \quad h_j(\mathbf{x}) = 0 \quad \text{for all } j = 1, \dots, n, \end{aligned} \quad (7.38)$$

where all functions $f(\mathbf{x})$ and $g_i(\mathbf{x})$ are convex functions, and all $h_j(\mathbf{x}) = 0$ are convex sets. In the following, we will describe two classes of convex optimization problems that are widely used and well understood.

7.3.1 Linear Programming

Consider the special case when all the preceding functions are linear, i.e.,

$$\begin{aligned} & \min_{\mathbf{x} \in \mathbb{R}^d} \mathbf{c}^\top \mathbf{x} \\ & \text{subject to } \mathbf{A}\mathbf{x} \leq \mathbf{b}, \end{aligned} \quad (7.39)$$

where $\mathbf{A} \in \mathbb{R}^{m \times d}$ and $\mathbf{b} \in \mathbb{R}^m$. This is known as a *linear program*. It has d variables and m linear constraints. The Lagrangian is given by

linear program
Linear programs are one of the most widely used approaches in industry.

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{c}^\top \mathbf{x} + \boldsymbol{\lambda}^\top (\mathbf{A}\mathbf{x} - \mathbf{b}), \quad (7.40)$$

where $\boldsymbol{\lambda} \in \mathbb{R}^m$ is the vector of non-negative Lagrange multipliers. Rearranging the terms corresponding to $\mathbf{x}$ yields

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = (\mathbf{c} + \mathbf{A}^\top \boldsymbol{\lambda})^\top \mathbf{x} - \boldsymbol{\lambda}^\top \mathbf{b}. \quad (7.41)$$

Taking the derivative of $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda})$ with respect to $\mathbf{x}$ and setting it to zero gives us

$$\mathbf{c} + \mathbf{A}^\top \boldsymbol{\lambda} = \mathbf{0}. \quad (7.42)$$

Therefore, the dual Lagrangian is $\mathcal{D}(\boldsymbol{\lambda}) = -\boldsymbol{\lambda}^\top \mathbf{b}$. Recall we would like to maximize $\mathcal{D}(\boldsymbol{\lambda})$. In addition to the constraint due to the derivative of $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda})$ being zero, we also have the fact that $\boldsymbol{\lambda} \geq \mathbf{0}$, resulting in the following dual optimization problem

It is convention to minimize the primal and maximize the dual.

$$\begin{aligned} & \max_{\boldsymbol{\lambda} \in \mathbb{R}^m} -\boldsymbol{\lambda}^\top \mathbf{b} \\ & \text{subject to } \mathbf{c} + \mathbf{A}^\top \boldsymbol{\lambda} = \mathbf{0} \\ & \quad \boldsymbol{\lambda} \geq \mathbf{0}. \end{aligned} \quad (7.43)$$

This is also a linear program, but with m variables. We have the choice of solving the primal (7.39) or the dual (7.43) program depending on